

Assignment 1: Snowflake Environment Setup (Beginner)

Objective: Understand Snowflake UI and basic objects.

Tasks

1. Log in to Snowflake (trial account).
2. Create:
 - A **Database**
 - A **Schema**
 - A **Virtual Warehouse**
3. Show all databases and schemas.
4. Suspend and resume the warehouse.

Deliverable:

- SQL script
- Screenshot of created objects

Create or replace Database Security_db;

Results (just now)			
Table Chart		Q 1 row 132ms	
status			
1	Database SECURITY_DB successfully created.		

Create or replace Schema security_db.security;

Results (just now)			
Table Chart		Q 1 row 54ms	
status			
1	Schema SECURITY successfully created.		

Create Virtual Warehouse:

CREATE or replace WAREHOUSE Security_wh

WITH

WAREHOUSE_SIZE = 'XSMALL'

AUTO_SUSPEND = 60

AUTO_RESUME = TRUE;

Results (just now)	
Table Chart 🔍 🔗 1 row 85ms 📄 🕒	
🔊	status
1	Warehouse SECURITY_WH successfully created.

Showing All Databases:

show databases;

Results (just now)								
Table Chart 🔍 🔗 7 rows 113ms 📄 🕒								
🔊	created_on	name	is_default	is_current	origin	owner	comment	opt
1	2025-12-10 08:16:13.607 -0800	CLI_DB	N	N		ACCOUNTADMIN		
2	2025-12-15 08:44:22.856 -0800	SECURITY_DB	N	Y		ACCOUNTADMIN		
3	2025-12-02 06:09:37.790 -0800	SNOWFLAKE	N	N	SNOWFLAKE.ACCOU			
4	2025-12-02 06:09:45.924 -0800	SNOWFLAKE_LEARNING_DB	N	N		ACCOUNTADMIN	Created by Snowflake	
5	2025-12-02 06:09:43.728 -0800	SNOWFLAKE_SAMPLE_DATA	N	N	SFSALESSHARED.SF	ACCOUNTADMIN	Preloaded TPC	
6	2025-12-09 00:26:21.721 -0800	TRAINING	N	N		ACCOUNTADMIN		
7	2025-12-03 21:31:05.725 -0800	USER\$HARIHARAN12	N	N				

Showing All Schemas:

show schemas;

Results (just now)							
Table Chart 🔍 🔗 3 rows 48ms 📄 🕒							
🔊	created_on	name	is_default	is_current	database_name	owner	comment
1	2025-12-15 08:47:04.306 -0800	INFORMATION_SCHEMA	N	N	SECURITY_DB		Views describing the contents of sct
2	2025-12-15 08:44:22.867 -0800	PUBLIC	N	N	SECURITY_DB	ACCOUNTADMIN	
3	2025-12-15 08:45:20.267 -0800	SECURITY	N	Y	SECURITY_DB	ACCOUNTADMIN	

-- Why We SUSPEND and Resume,Because Suspend is a cost save, Resume is query run

-- Suspending a warehouse stops compute usage to save cost, and resuming it restarts the warehouse to run queries.

Suspend the warehouse:

```
ALTER WAREHOUSE security_wh SUSPEND;
```

Resume the warehouse:

```
ALTER WAREHOUSE security_wh RESUME;
```

Assignment 2: User, Role & Access Control (RBAC)

Objective: Learn Snowflake security and role hierarchy.

Tasks

1. Create roles:
 - STUDENT_ROLE
 - INSTRUCTOR_ROLE
2. Create users:
 - student1
 - instructor1
3. Assign:
 - STUDENT_ROLE → student1
 - INSTRUCTOR_ROLE → instructor1
4. Grant permissions:
 - Student → **SELECT** on tables
 - Instructor → **SELECT, INSERT**
5. Verify access using SHOW GRANTS.

Deliverable:

- SQL commands

Create Student_role and instructor_role:

```
create role student_role;
```

Results (just now)	
Table	Chart
1 row 53ms	
1	Role STUDENT_ROLE successfully created.

```
create role instructor_role;
```

Results (just now)	
Table	Chart
1 row 56ms	
1	Role INSTRUCTOR_ROLE successfully created.

Create Users:

Create User student1

password='student123'

Must_change_password=True;

Results (just now)	
Table	Chart
1 row 212ms	
status	
1	User STUDENT1 successfully created.

CREATE or replace user instructor1

password='INSTRUCTOR123'

must_change_password=True;

Results (just now)	
Table	Chart
1 row 215ms	
status	
1	User INSTRUCTOR1 successfully created.

Assign the roles to Users:

GRANT role student_role to user student1;

GRANT role instructor_role to user instructor1;

Grant permissions:

-- Student role: SELECT only

GRANT SELECT

ON TABLE security_db.security.students

TO ROLE STUDENT_ROLE;

-- Instructor role: SELECT and INSERT

GRANT SELECT, INSERT

ON TABLE security_db.security.students

TO ROLE INSTRUCTOR_ROLE;

SHOW GRANTS TO USER student1;

The screenshot shows a database query results window with the title 'Results (just now)'. It has a 'Table' tab selected. The results are displayed in a table with 10 columns: 'id', 'created_on', 'privilege', 'granted_on', 'name', 'role', 'granted_to', 'grantee_name', 'grant_option', and 'granted_by'. There is one row of data.

id	created_on	privilege	granted_on	name	role	granted_to	grantee_name	grant_option	granted_by
1	2025-12-15 11:29:35	USAGE	ROLE	STUDENT_ROLE	STUDENT_ROLE	USER	STUDENT1	false	SECURITYADMIN

SHOW GRANTS TO USER instructor1;

The screenshot shows a database query results window with the title '(just now) ... x'. It has a 'Table' tab selected. The results are displayed in a table with 10 columns: 'id', 'created_on', 'privilege', 'granted_on', 'name', 'role', 'granted_to', 'grantee_name', 'grant_option', and 'granted_by'. There is one row of data.

id	created_on	privilege	granted_on	name	role	granted_to	grantee_name	grant_option	granted_by
1	2025-12-15 11:29:41	USAGE	ROLE	INSTRUCTOR_ROLE	INSTRUCTOR_ROLE	USER	INSTRUCTOR1	false	SECURITYADMIN

Short explanation of RBAC

1. Access is given to **roles**, not directly to users.
2. Users get permissions by **assigning roles** to them.
3. It helps keep data **secure and easy to manage**.

Assignment 3: Table Creation & Data Loading

Objective: Work with structured data.

Tasks

1. Create a table:

CREATE TABLE students (

id INT,

name STRING,

course STRING,

score INT

);

2. Insert at least **10 records**.
3. Query:
 - All students
 - Students with score > 80
4. Update one record.
5. Delete one record.

Deliverable:

- SQL file with queries

- Output screenshots

Create a table:

```
CREATE TABLE security_db.security.students (  
    id INT,  
    name varchar(20),  
    course varchar(20),  
    score INT  
);
```



The screenshot shows a database interface with a query results window. The window title is "Results (just now)". It has two tabs: "Table" (selected) and "Chart". The results are displayed in a table with one row and one column labeled "status". The row contains the text "Table STUDENTS successfully created.".

	status
1	Table STUDENTS successfully created.

Insert at least 10 records:

```
INSERT INTO students (id, name, course, score) VALUES  
  
(1, 'Arun', 'Data Engineering', 85),  
(2, 'Priya', 'Data Science', 92),  
(3, 'Karthik', 'Snowflake', 78),  
(4, 'Divya', 'Python', 88),  
(5, 'Ramesh', 'SQL', 90),  
(6, 'Suresh', 'Data Engineering', 72),  
(7, 'Meena', 'Snowflake', 95),  
(8, 'Vijay', 'Python', 80),  
(9, 'Anitha', 'Data Science', 84),  
(10, 'Rahul', 'SQL', 76),  
(11, 'Deepa', 'Snowflake', 89),  
(12, 'Ganesh', 'Data Engineering', 91),  
(13, 'Lakshmi', 'Python', 83),  
(14, 'Manoj', 'SQL', 68),  
(15, 'Kavya', 'Data Science', 87),  
(16, 'Naveen', 'Snowflake', 93),  
(17, 'Pooja', 'Python', 79),
```

(18, 'Sathish', 'Data Engineering', 82),
 (19, 'Aishwarya', 'SQL', 88),
 (20, 'Hari', 'Data Science', 90),
 (21, 'Bala', 'Data Engineering', 55),
 (22, 'Keerthi', 'Data Science', 48),
 (23, 'Mohan', 'Snowflake', 52),
 (24, 'Swathi', 'Python', 45),
 (25, 'Prakash', 'SQL', 58),
 (26, 'Renuka', 'Data Engineering', 49),
 (27, 'Sanjay', 'Snowflake', 54),
 (28, 'Nithya', 'Python', 50),
 (29, 'Ajith', 'Data Science', 47),
 (30, 'Revathi', 'SQL', 56),
 (31, 'Aravind', 'Snowflake', 53),
 (32, 'Kavitha', 'Data Engineering', 44),
 (33, 'Kumar', 'Python', 59),
 (34, 'Sneha', 'SQL', 46),
 (35, 'Vinoth', 'Data Science', 51),
 (36, 'Preethi', 'Snowflake', 42),
 (37, 'Raghul', 'Python', 57),
 (38, 'Malathi', 'Data Engineering', 48),
 (39, 'Saravanan', 'SQL', 55),
 (40, 'Nandhini', 'Data Science', 49);

(just now ... x)	
Table Chart 🔍 🗑️ 1 row ⓘ 406ms 📊 ⬇️ ⌚	
🗑️ # number of rows inserted	
1	40

Show all Students:

select*from students;

Results (just now)

TableChart

40 rows375ms

# ID	NAME	COURSE	SCORE
1	Aishwarya	Data Engineering	85
2	Ajith	Data Science	92
3	Karthik	Snowflake	78
4	Divya	Python	88
5	Ramesh	SQL	90
6	Suresh	Data Engineering	72
7	Meena	Snowflake	95
8	Vijay	Python	80
9	Anitha	Data Science	84
10	Rahul	SQL	76
11	Deepa	Snowflake	89
12	Ganesh	Data Engineering	91
13	Lakshmi	Python	83
14	Manoj	SQL	68
15	Kavya	Data Science	87
16	Naveen	Snowflake	93
17	Pooja	Python	79
18	Sathish	Data Engineering	82
19	Aishwarya	SQL	88
20			
21			
22	Keerthi	Data Science	48
23	Mohan	Snowflake	52
24	Swathi	Python	45
25	Prakash	SQL	58
26	Renuka	Data Engineering	49
27	Sanjay	Snowflake	54
28	Nithya	Python	50
29	Ajith	Data Science	47
30	Revathi	SQL	56
31	Aravind	Snowflake	53
32	Kavitha	Data Engineering	44
33	Kumar	Python	59
34	Sneha	SQL	46
35	Vinoth	Data Science	51
36	Preethi	Snowflake	42
37	Raghul	Python	57
38	Malathi	Data Engineering	48
39	Saravanan	SQL	55
40	Nandhini	Data Science	49

Query:

```
select *from students where score >80;
```

Results (just now)

Table Chart 14 rows 391ms

# ID	NAME	COURSE	SCORE
1	Aishwarya	Data Science	82
2	Anitha	Data Engineering	85
3	Arjun	Data Science	92
4	Divya	Python	88
5	Ramesh	SQL	90
6	Meena	Snowflake	95
7	Anitha	Data Science	84
8	Deepa	Snowflake	89

Results (just now)

Table Chart 14 rows 391ms

# ID	NAME	COURSE	SCORE
8	Ganesh	Data Engineering	91
9	Lakshmi	Python	83
10	Kavya	Data Science	87
11	Naveen	Snowflake	93
12	Sathish	Data Engineering	82
13	Aishwarya	SQL	88
14	Hari	Data Science	90

Update one record:

```
UPDATE students
```

```
SET name = 'Hari',course = 'Snowflake'
```

```
WHERE id = 1;
```

Results (just now)

Table Chart 1 row 764ms

# number of rows updated	# number of multi-joined rows updated
1	0

1st Record Name and course changed

Results (just now)

Table Chart 40 rows 127ms

# ID	NAME	COURSE	SCORE
1	Hari	Snowflake	85
2	Priya	Data Science	92
3	Karthik	Snowflake	78
4	Divya	Python	88
5	Ramesh	SQL	90

Delete the one record:

DELETE FROM students WHERE id = 2;

(just now) ... x

Table Chart

1 row 736ms

#	number of rows deleted
1	1

Results (just now)

Table Chart

39 rows 422ms

#	ID	NAME	COURSE	SCORE
1	1	Hari	Snowflake	85
2	3	Karthik	Snowflake	78
3	4	Divya	Python	88
4	5	Ramesh	SQL	90
5	6	Suresh	Data Engineering	72

Assignment 4: File Loading using Stage & COPY (Intermediate)

Objective: Learn Snowflake data ingestion.

Tasks

1. Create an **internal stage**.
2. Upload a CSV file.
3. Use COPY INTO to load data.
4. Validate loaded records.
5. Handle one data error using VALIDATION_MODE.

Deliverable:

- SQL script
- CSV file
- Query output

Create an internal stage:

Create Stage security_stg;

Results (just now)	
Table	Chart
1 row	188ms
status	
1	Stage area SECURITY_STG successfully created.

Upload a CSV file:

Upload using UI:

Go to Snowflake UI

Databases → Schema → Stages

Select stage → Upload file

Why we use UI:

Easy and visual

Good for **learning, testing, demos**

Small files

Upload using CMD / CLI (PUT command):

PUT file://C:/data/students.csv @security_stg;

Why we use:

Faster

Can upload large files

Can automate (scripts, jobs)

Upload Your Files

Drag and drop files here

or

[Browse](#)

File size limit: 250MB

Select database, schema and stage

No Database selected Select Stage

Specify the path to an existing folder or create a new one (optional)

Example / Path / To / Folder

Once the files finish uploading, your Directory Table will automatically refresh.

Close Upload

Use COPY INTO to load data:

```
COPY INTO students
FROM @security_stg/security.csv
FILE_FORMAT = (
TYPE = 'CSV'
SKIP_HEADER = 1
);
```

Results (just now)	
Table	Chart
1 row 674ms	
#	status
1	Copy executed with 0 files processed.

Validate loaded records:

checking records

```
SELECT COUNT(*) FROM students;
```

Results (just now)	
Table	Chart
1 row 54ms	
#	COUNT(*)
1	39

Handle one data error using VALIDATION_MODE:

```
COPY INTO students
FROM @security_stg/students_error.csv
FILE_FORMAT = (
TYPE = 'CSV'
SKIP_HEADER = 1
)
VALIDATION_MODE = 'RETURN_ERRORS';
```

Results (just now)									
Table	Chart	1 row 1.6s							
#	ERROR	FILE	# LINE	# CHARACTER	# BYTE_OFFSET	CATEGORY	CODE	SQL_STA	
1	Numeric value 'ABC' is	students_error.csv	3	16	56	conversion	100038	22018	

Assignment 5: Time Travel & Data Recovery

Objective: Understand Snowflake Time Travel.

Tasks

1. Delete records from a table.
2. Query data using **Time Travel**.
3. Restore deleted data.
4. Explain how Time Travel helps in recovery.

Deliverable:

- SQL queries
 - Short explanation
-

Assignment 6: Warehouses & Performance

Objective: Learn compute management.

Tasks

1. Create:
 - SMALL_WH
 - MEDIUM_WH
2. Run same query on both warehouses.
3. Compare execution time.
4. Suspend unused warehouse.

Deliverable:

- Query history screenshot

Create SMALL warehouse:

```
CREATE or replace WAREHOUSE SMALL_WH  
WITH  
WAREHOUSE_SIZE = 'SMALL'  
AUTO_SUSPEND = 60  
AUTO_RESUME = TRUE;
```


USE WAREHOUSE SMALL_WH;

Results (just now)

Table Chart 1 row 54ms

status
Statement executed successfully.

SELECT movie_name, genre,
DENSE RANK () OVER (PARTITION BY genre ORDER BY rating DESC) AS rating_movies
FROM movies;

Results (just now)

Table Chart 25,000,095 rows 64ms

	MOVIE_NAME	GENRE	RATING_MOVIES
1	History of Future Folk, The (2012)	Adventure Comedy Musical Sci-Fi	1
2	History of Future Folk, The (2012)	Adventure Comedy Musical Sci-Fi	1
3	History of Future Folk, The (2012)	Adventure Comedy Musical Sci-Fi	1
4	History of Future Folk, The (2012)	Adventure Comedy Musical Sci-Fi	1
5	History of Future Folk, The (2012)	Adventure Comedy Musical Sci-Fi	1

Query History

use warehouse medium_wh;

Results (just now)

Table Chart 1 row 53ms

status
Statement executed successfully.

SELECT course, COUNT(*) AS total_students
FROM security_db.security.students
GROUP BY course;

Results (just now)

Table Chart 25,000,095 rows 26ms

	MOVIE_NAME	GENRE	RATING_MOVIES
1	History of Future Folk, The (2012)	Adventure Comedy Musical Sci-Fi	1
2	History of Future Folk, The (2012)	Adventure Comedy Musical Sci-Fi	1
3	History of Future Folk, The (2012)	Adventure Comedy Musical Sci-Fi	1
4	History of Future Folk, The (2012)	Adventure Comedy Musical Sci-Fi	1
5	History of Future Folk, The (2012)	Adventure Comedy Musical Sci-Fi	1

SMALL_WH → Loads slower, suitable for small files
MEDIUM_WH → Loads large files (like 500 MB) faster

Medium_wh:

Details

Status

Success

Duration

26ms

Driver Status

N/A

Start Time

12/17/2025, 12:33:16 AM

Query ID

01c11837-0001-6deb-000c-ee9600043d8a

Client Driver

Snowsight

End Time

12/17/2025, 12:33:16 AM

Query Tag

—

Session ID

3640027733184754

Warehouse Size

—

Attributed Compute Credits

—

Parameterized query hash

49e37b4a806a0b7c80e087f381f40377

small_wh:

Details

Status

Success

Duration

30ms

Driver Status

N/A

Start Time

12/17/2025, 12:41:57 AM

Query ID

01c1183f-0001-6deb-000c-ee960004602e

Client Driver

Snowsight

End Time

12/17/2025, 12:41:57 AM

Query Tag

—

Session ID

3640027733184754

Warehouse Size

—

Attributed Compute Credits

—

Parameterized query hash

49e37b4a806a0b7c80e087f381f40377

Performance comparison notes:

- The same query was executed on **SMALL** and **MEDIUM** warehouses.
- The **MEDIUM warehouse** completed the query faster due to higher compute resources.
- The **SMALL warehouse** took more execution time because of limited compute.
- Larger warehouses provide better performance for large data and heavy workloads.
- Suspending unused warehouses helps reduce compute cost.

Suspend unused warehouse:

ALTER WAREHOUSE SMALL_WH suspend;

Results (just now)	
Table	Chart
1 row 48ms	
status	
1	Statement executed successfully.

SHOW WAREHOUSES LIKE 'SMALL_WH';

Results (just now)

TableChart

1 row

123ms

	A name	A state	A type	A size	# min_cluster_count	# max_cluster_count	# sta
1	SMALL_WH	SUSPENDED	STANDARD	Small	1	1	

ALTER WAREHOUSE SMALL_WH resume;

Results (just now)	
Table	Chart
1 row 53ms	
1	Statement executed successfully.

SHOW WAREHOUSES LIKE 'SMALL_WH';

Results (just now)

TableChart

1 row84ms

	<u>A</u> name	<u>A</u> state	<u>A</u> type	<u>A</u> size	# min_cluster_count	# max_cluster_count	# sta
1	SMALL_WH	STARTED	STANDARD	Small	1	1	

Assignment 7: Views & Secure Views

Objective: Control data visibility.

Tasks

1. Create a **View** hiding sensitive columns.
2. Create a **Secure View**.
3. Grant view access to Student role.
4. Test access using Student role.

Deliverable:

- SQL queries
- Explanation of Secure View

Create a View hiding sensitive columns:

CREATE OR REPLACE VIEW student_view AS

SELECT id,name,

course -- score column is hide

FROM security_db.security.students;

Results (just now)	
TableChart Q 1 row 164ms	
	status
1	View STUDENT_VIEW successfully created.

Create a **Secure View**:

CREATE OR REPLACE SECURE VIEW student_secure_view AS

SELECT id, name, course

FROM security_db.security.students;

(just now) X	
TableChart Q 1 row 145ms	
	status
1	View STUDENT_SECURE_VIEW successfully created.

Grant view access to Student role:

GRANT SELECT

ON VIEW security_db.security.student_secure_view

TO ROLE STUDENT_ROLE;

(just now) X	
TableChart Q 1 row 145ms	
	status
1	View STUDENT_SECURE_VIEW successfully created.

Assignment 8: Scaling & Auto Suspend (Advanced)

Objective: Understand Snowflake scaling behavior.

Tasks

- 1. Enable **auto-suspend** and **auto-resume**.
- 2. Configure **multi-cluster warehouse**.
- 3. Simulate concurrent queries.
- 4. Observe scaling behavior.

Deliverable:

- Warehouse configuration
- Query history evidence

Enable **auto-suspend** and **auto-resume**:

```
ALTER WAREHOUSE SMALL_WH
SET
  AUTO_SUSPEND = 60,
  AUTO_RESUME = TRUE;
```

Results (just now)	
TableChart 1 row79ms	
	status
1	Statement executed successfully.

```
ALTER WAREHOUSE MEDIUM_WH
SET
  AUTO_SUSPEND = 60,
  AUTO_RESUME = TRUE;
```

Results (just now)	
Table	Chart
1 row	79ms
id	status
1	Statement executed successfully.

Configure multi-cluster warehouse:

Multi-cluster warehouses automatically scale compute resources to handle concurrent queries efficiently without manual intervention.

CREATE OR REPLACE WAREHOUSE MC_WH

WITH

WAREHOUSE_SIZE = 'MEDIUM'

WAREHOUSE_TYPE = 'STANDARD'

MIN_CLUSTER_COUNT = 1

MAX_CLUSTER_COUNT = 3

SCALING_POLICY = 'STANDARD'

AUTO_SUSPEND = 60

AUTO_RESUME = TRUE;

Results (just now)	
Table	Chart
1 row	149ms
id	status
1	Warehouse MC_WH successfully created.

Simulate concurrent queries:

Concurrent queries were simulated by running multiple queries at the same time using different worksheets.

USE WAREHOUSE MC_WH;

SELECT COUNT(*) FROM security_db.security.students;

SELECT COUNT(*) FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF100.ORDERS;

SELECT COUNT(*) FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF100.CUSTOMER

Observe scaling behaviour:

CREATE OR REPLACE WAREHOUSE MC_WH

WITH

WAREHOUSE_SIZE = 'MEDIUM'

WAREHOUSE_TYPE = 'STANDARD'

MIN_CLUSTER_COUNT = 1

MAX_CLUSTER_COUNT = 3

SCALING_POLICY = 'STANDARD'

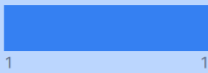
AUTO_SUSPEND = 60

AUTO_RESUME = TRUE;

Results (1 minute ago)

TableChart

556 rows3.3s

	QUERY_ID	START_TIME	WAREHOUSE_NAME	CLUSTER_NUMBER
	01c11a58-0001-6e16-000c-ee96... 0.2% 01c11a61-0001-6deb-000c-ee96... 0.2% +98 more		MC_WH 100.0%	
1	01c11ab0-0001-6e16-000c-ee9600047a76	2025-12-16 21:36:54.957 -0800	MC_WH	null
2	01c11ab0-0001-6deb-000c-ee9600046bbe	2025-12-16 21:36:54.945 -0800	MC_WH	null
3	01c11ab0-0001-6deb-000c-ee9600046bc2	2025-12-16 21:36:54.945 -0800	MC_WH	null
4	01c11ab0-0001-6e16-000c-ee9600047a72	2025-12-16 21:36:54.942 -0800	MC_WH	null
5	01c11ab0-0001-6deb-000c-ee9600046bb6	2025-12-16 21:36:54.926 -0800	MC_WH	null
6	01c11ab0-0001-6deb-000c-ee9600046bb2	2025-12-16 21:36:54.908 -0800	MC_WH	null

Sum 4 Avg 1